Project 11 (Bonus) — MCPCrew-Multi-Agent-Research-Team-Powered-By-MCP-Servers

Type: Multi-agent (4 agents: 1 Supervisor + 3 specialists) · **Stack:** Python, LangGraph (`Command`), `mcp` (official MCP SDK), OpenAI SDK, asyncio, Pydantic · **Level:** Advanced

Tagline: A Supervisor-led research team where each specialist owns its *own* MCP server connection, synthesizes one report through a Writer, and keeps working when one specialist's server is down.

> Not part of the numbered Doc01-19 arc — a bonus/portfolio project built on 06_tools_function_calling's MCP sections and 11_multi_agent_systems's Supervisor pattern. Combines Project 8's MCP client with Project 4's `Command` Supervisor.

Charter (what this project is)

Build a Supervisor-led team of specialists where two of them each open their *own* MCP server connection, and a Writer synthesizes their findings into one report. The point isn't "one agent with three MCP servers" (Project 8 already did that) — it's that in a real company, different capabilities are owned by different teams with different servers and different uptimes. So each specialist owns a narrow job and its own connection, the Supervisor only *routes* (never touches a connection itself), and one specialist's dead server degrades the team instead of crashing it.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4

The Story — what you're actually building

MCPCrew is asked for a short status report on an internal project, "Project Atlas." It has two sources: a **notes** service (someone's `.md` notes) and a **web** service (fetch a known status page) — each a separate MCP server, owned by a different "team." A **Notes Agent** and a **Web Agent** each own one connection. A **Supervisor** decides, per request, who runs. A **Writer** turns their findings into one coherent report, not two raw dumps side by side.

Step 1 proves each specialist works completely alone, MCP connection and all. Step 2 adds the Supervisor (Project 4's `Command` pattern) routing between the two proven specialists. Step 3 adds the Writer so the output is one report. Step 4 asks the hardest, most realistic question: what does the team do when one specialist's server just isn't there? (Answer: finish the report from the other specialist's findings, and *say* the web page couldn't be reached — not crash, not silently pretend.)

> **Before you read further — think about it yourself:** why build three specialist agents with three connections instead of one agent holding all three MCP servers (Project 8 Step 3)? And when the Web server is down, where exactly should the failure be caught — in the Supervisor, in the graph, or inside the Web Agent's own node — so the rest of the team keeps working? Settle both before Step 1.

Where This Fits

A bonus/portfolio project that combines two real patterns you've already built separately: a standard tool protocol (MCP, Project 8) and a coordinated multi-agent team (Supervisor + `Command`, Project 4). What's new is putting *several* agents, each with its *own* MCP server, under one routing layer — the realistic shape of a company's "assistant team" where different capabilities are owned by different services.

Project Structure (the professional layout)

`main.py` at the repo root, all code in `src/`. The MCP servers live in `servers/` because the specialists launch them as subprocesses, not import them.

The final layout (after Step 4)

```
<repo folder>/   (MCPCrew-Multi-Agent-Research-Team-Powered-By-MCP-Servers/)
├── servers/
│   ├── notes_server.py         MCP server: add_note, search_notes
│   └── web_server.py           MCP server: fetch_page (mocked)
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── notes_agent.py      run_notes_agent() -> findings
│   │   ├── supervisor.py       decide_next(), supervisor() (Command)
│   │   ├── web_agent.py        run_web_agent() -> (findings, unavailable)
│   │   └── writer_agent.py     run_writer() -> report
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              create_client()
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
```

```
│   ├── graph.py                build_graph(client)
│   ├── mcp_client.py           run_specialist(): the shared MCP ReAct loop
│   └── state.py                TeamState
├── tests/
│   ├── integration/
│   │   └── test_live_crew.py   real servers + model
│   └── unit/
│       ├── test_config.py
│       └── test_supervisor.py  routing on hand-made states (no model)
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds config/exceptions/logger, `models/llm`, `mcp_client.run_specialist`, the two servers, `agents/notes_agent` + `agents/web_agent`, `main.py` (standalone), `test_config`.
- **Step 2** adds `state.py`, `agents/supervisor.py` (`Command`), `graph.py` (supervisor routes to notes/web nodes), and `test_supervisor`.
- **Step 3** adds `agents/writer_agent.py` and a `writer` node.
- **Step 4** adds the degradation path in `web_agent` (a dead server → a clear "unavailable" result) and the Writer reading `web_unavailable`.

Setup (do this once, before Step 1)

```
cd MCPCrew-Multi-Agent-Research-Team-Powered-By-MCP-Servers
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install mcp openai langgraph python-dotenv pydantic
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. This project is `asyncio`-based wherever it talks to MCP (like Project 8).

A Real Example (so this isn't just theory)

Scenario: a status report on "Project Atlas", from a **notes** server and a **web** server (mocked fetch).

1. `"What do my notes say about Project Atlas?"` — Notes Agent only.
2. `"Fetch the Atlas status page and summarize it."` — Web Agent only.
3. `"Check my notes for Atlas, fetch its status page too, and give one combined report."` — both specialists, then the Writer.
4. Task 3 with the **web server deliberately not started** — the team still produces a report from the notes alone, clearly saying the web page couldn't be reached.

How testing works in this project

Two layers, one style. Because this builds on Doc06/11 (plain test scripts, not `pytest`), **every test is a plain OK/WRONG script**, run with `python -m`:

- **`tests/unit/`** — no model, no servers: `test_config`, `test_supervisor` (routing on hand-made states).
- **`tests/integration/test_live_crew.py`** — launches the real MCP servers and the model: a both-specialist report; and the web-down case producing a degraded report that names the gap.

Each step ends with a **Tests for this step** box.

Step 1 — Two Standalone MCP-Connected Specialists

What this step does: the Notes Agent and Web Agent, each connecting to its own MCP server, each proven alone — no Supervisor, no graph yet.

Why this step matters: prove the connection before adding complexity. A broken connection debugged inside Step 2's graph means debugging two things at once.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `servers/notes_server.py` (an `MCPServer` with `add_note`, `search_notes`, seeded with a few Atlas notes) and `servers/web_server.py` (one `fetch_page` tool over a mocked URL→text dict; note at the top that a real deploy would use a real fetch server). 2. Write `src/mcp_client.py`: `run_specialist(client, command, args, task)` — open a stdio connection to that server, discover its tools, run a small ReAct loop (reuse Project 8's tool-bridge + loop), return the findings text. (This is shared by both specialists.) 3. Write `agents/notes_agent.py` (`run_notes_agent` → calls `run_specialist` with the notes server) and `agents/web_agent.py` (`run_web_agent` with the web server). 4. Write `main.py` to run both standalone on tasks 1–2 and print the results.

Your files after Step 1: config, exceptions, `utils/logger`, `models/llm`, `mcp_client`, `agents/notes_agent`, `agents/web_agent`, `servers/*`, `main.py`, `tests/unit/test_config.py`.

Tests for this step (`python -m tests.unit.test_config`): missing/empty setting raises; a present key loads.

Step 2 — A Supervisor Routes Between Them

What this step does: wraps the two proven specialists in a LangGraph graph with a `Command` -based Supervisor (Project 4's pattern) that decides who runs per request.

Why this step matters: the Supervisor coordinates without owning any connection — it only routes to a specialist node and reads its result from shared state, exactly like Project 4's Supervisor.

What's new vs. Step 1: `state.py` , `agents/supervisor.py` , `graph.py` . **What stays the same:** the specialists.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `state.py` (`TeamState` : `task` , `notes_findings` , `web_findings` , `web_unavailable` , `report` , `routing_log`). 2. Write `agents/supervisor.py` : `decide_next(state)` (route to `notes` / `web` / `writer` based on what's still needed) and `supervisor(state)` returning `Command(goto=target, update={routing_log})` . 3. Write `graph.py` : `START → supervisor` ; `notes_node` and `web_node` are **async** nodes that call the specialists and return `Command(goto="supervisor", update=...)` . Invoke with `.ainvoke()` .

Your files after Step 2: + `state.py` , `agents/supervisor.py` , `graph.py` , `tests/unit/test_supervisor.py` .

Tests for this step (`python -m tests.unit.test_supervisor`): a fresh "both" task routes to `notes` first; with notes done it routes to `web` ; with both done it routes to `writer` .

Step 3 — The Writer Synthesizes a Final Report

What this step does: adds a Writer (a plain LLM call, no tools) that takes both specialists' findings and writes **one** coherent report.

Why this step matters: without it, the output is two raw specialist dumps side by side. The Writer's prompt must say *synthesize*, not concatenate.

What's new vs. Step 2: `agents/writer_agent.py` and a `writer` node. **What stays the same:** the specialists and the Supervisor.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `agents/writer_agent.py` : `run_writer(client, task, notes_findings, web_findings, web_unavailable)` — a prompt that synthesizes the findings into one report, and that explicitly notes any unavailable source. 2. Add a `writer_node` to `graph.py` ; the Supervisor

routes to it once the needed specialists have run. 3. Run task 3 — confirm the report reads as one synthesized status update, not two paragraphs stapled together.

Your files after Step 3: + `agents/writer_agent.py`, `graph.py` (writer node).

Tests for this step: the integration test's both-specialist case (the report mentions both sources).

Step 4 — Graceful Degradation When One MCP Server Is Down

What this step does: makes the team survive a specialist's server being down — the Web Agent catches the failure inside its own node, writes a clear "unavailable" result, and the Writer says so in the report.

Why this step matters: production servers go down. A team that can't tell "one source is gone" from "everything broke" isn't trustworthy.

What's new vs. Step 3: a `try/except` + `asyncio.wait_for` in `web_agent`, a `web_unavailable` flag, and the Writer reading it. **What stays the same:** every specialist that *does* connect works as before.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. In `web_agent` (via `run_specialist`), wrap the connect + `initialize()` in `asyncio.wait_for(..., timeout=...)` inside `try/except`. On failure, return `("", True)` — empty findings, `unavailable=True` — and log it, **inside the node**, never letting the exception escape. 2. Make `web_node` write `web_unavailable` into state, and `run_writer` read it and say "the web status page could not be reached." 3. Run task 3 with the web server's launch command deliberately broken — confirm a report from the notes alone that names the gap.

Your files after Step 4 (final): the full layout above.

Tests for this step: the integration test's web-down case (`web_unavailable` true; the report names the gap; the notes findings are still present).

Final Deliverable: a Supervisor-led team where two specialists each own their own MCP connection, a Writer synthesizes one report, and one dead server degrades the team instead of crashing it.

Run, Test and Configure the Final Project

```
python main.py
python -m tests.unit.test_config
python -m tests.unit.test_supervisor
python -m tests.integration.test_live_crew
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity

Checklist Before You Call This Done

- [ ] Each specialist was proven standalone (own MCP connection) before the graph existed
- [ ] The Supervisor only routes — it never touches a `ClientSession` or `stdio_client`
- [ ] A both-specialist task produces one synthesized report, not two raw dumps
- [ ] A dead server is caught **inside** the Web Agent's node; the graph doesn't crash
- [ ] The report explicitly says a source was unavailable when it was
- [ ] Specialist nodes are `async` and the graph is run with `.ainvoke()` consistently

Publish It to GitHub

Name the repo `MCPCrew-Multi-Agent-Research-Team-Powered-By-MCP-Servers`, add the `README.md`, commit `.env.example` (never `.env`), and push. It shows you can combine a standard tool protocol with a coordinated multi-agent team, with real resilience.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full code only if you say **"Show me the solution."**